

DuplexAgent: Recursive Harness Improvement for Full-Duplex Voice Agent Collaboration

Yingda Shen* Yuxiang Wang* Kunyu Feng* Qinke Ni* Jiaqi Li*[†]
Minghao Hsu* Junan Zhang* Dekun Chen* Yutong Bian*[†] Zhizheng Wu*[†]

* The Chinese University of Hong Kong, Shenzhen

[†] Amphion Technology Co., Ltd

[Project Page](#) [GitHub](#)

Abstract

Spoken agents are converging on a collaboration pattern: a full-duplex interaction model stays on the live channel as the entry to the conversation, while search, reasoning, and coding are handled through asynchronous delegation. A duplex model supports continuous listening and speaking, but complex reasoning and tool use may exceed its capabilities. A coding agent can plan and execute extended tasks, but its sequential interface is a poor fit for live conversation. Combining them requires a harness that coordinates task acceptance, progress, cancellation, replacement, and result delivery while keeping the conversation responsive. Existing harnesses often rely on coupled heuristics, making them difficult to improve systematically from evidence. We present DuplexAgent, a full-duplex collaboration system whose harness expresses this workflow as six editable modules, and *Duplex-Harness-RSI*, a closed loop that revises them from interaction traces. A simulator automatically generates timed test conversations, runs the system, and produces failure traces that identify the collaboration modules requiring repair. Reasoning LLMs and coding agents in the delegation pool also serve the improvement loop: the Exam Planner selects the next tests from observed weaknesses and repair history, and the Harness Editor proposes targeted module changes. The capabilities that serve the user thus also improve the system’s coordination. Experiments on intelligence, agentic, and duplex benchmarks show that DuplexAgent combines continuous interaction with difficult reasoning and complex task execution, achieving stronger spoken-knowledge and executable-tool scores than the compared delegated systems while maintaining strong interruption response. A harness ablation further shows that this modular, verifiable RSI loop outperforms the initial heuristic harness and ordinary iterative editing.



Figure 1: Selected system results on MMSU, BFCL parallel-multiple, FDB-v3 Pass@1, and FDB-v1 interrupt TOR. DuplexAgent and Qwen-Audio-Agent use the Qwen-Realtime attachment. Full results appear in Tables 4 and 5.

1 Introduction

As language-model agents move from answering questions to carrying out work such as search, tool use, multi-step plans, and code edits (Yao et al., 2023), users need to supervise that work as it proceeds: they refine the goal, ask for status, interrupt an answer, or start a second task before the first has finished. Real-time speech provides a low-friction interface for this supervision, but it replaces orderly text turns with overlap, backchannels, asides, and changes of mind. Continuous interaction and complex task completion are usually handled by different model families. A full-duplex interaction model keeps listening and speaking on the live timeline, supporting turn-taking, barge-in, and backchannels (Défossez et al., 2024; Wang et al., 2024). A long search, difficult reasoning pass, or repository edit, however, requires deliberate work beyond the conversation. Tool-using and coding agents can perform that work (Yao et al., 2023), but a sequential agent cannot remain attentive to the live channel while it runs.

Recent systems bridge this division by keeping the interaction model on the live channel and assigning demanding work to a delegation pool through asynchronous handoff (Deng et al., 2026; Ant Group (Venus Team) and Tsinghua University, 2026). This combines continuous interaction with the reasoning ability of LLMs and the long-horizon execution of coding agents, but does not determine how an active task should be updated, cancelled, or returned. Suppose a user requests last night’s match result, then says “actually, the women’s match” while the first search is running, and begins another utterance just as the original result returns. The system must revise the active task, reject the obsolete result, and wait for a valid speaking opportunity. A fixed coordinator may instead launch a second independent search, speak the first result over the user, or later cancel the wrong task. Existing systems implement useful instances of handoff, acknowledgement, and delivery control (Deng et al., 2026; Ant Group (Venus Team) and Tsinghua University, 2026; Orantqing et al., 2026; Lin et al., 2025c), but these decisions are generally coupled in system-specific heuristics. When the same failure recurs, a trace may not clearly identify whether delegation, task interpretation, background execution, or delivery should change. A bounded repair is then difficult to isolate, evaluate, and transfer.

We present *DuplexAgent*, a full-duplex collaboration system whose harness makes this control explicit in six editable modules: Policy determines when work is delegated; Intake interprets requests and task control; Hold maintains the conversation while work is pending; Worker selects and instructs a delegate; Deliver controls when and how a result is returned; and Memory preserves shared context. The interaction model remains the conversational entry, while a *delegation pool* of reasoning LLMs and coding agents carries delegated work. A common harness coordinates their exchange through model-specific adapters, supporting both open-weight and hosted interaction models and one or several delegates.

Separating these responsibilities gives each failure a place to be investigated and provides the basis for *Duplex-Harness-RSI*. In the match-query example, the interaction trace can distinguish a task that was not updated from an obsolete result that was incorrectly delivered. The diagnosis points to Intake, Deliver, or a repair involving both, rather than requiring a change to the entire coordination policy. The proposed change is then specific enough to evaluate against the failed conversation and other test conditions. Repeated across traces, the same process becomes a loop of observation, diagnosis, editing, and evaluation, using earlier outcomes to guide the next revision.

Each test in the loop is a timed conversation assembled from a request type and an interaction pattern. It specifies expected behaviour; the resulting trace supports attribution to the responsible module. After each round, a reasoning LLM from the same delegation pool that serves users acts as the *Exam Planner*: it analyses performance, attributed failures, and repair history, then adjusts the coverage of the next exam. The simulator generates the conversations; scripts execute and score them; and a coding agent acts as the *Harness Editor*, proposing changes to the modules named by the diagnosis. Fixed evaluation code compares each candidate with the incumbent on paired runs and checks disjoint protection cases before retention.

Section 2 reviews related work. Section 3 presents the six modules, test generation, failure attribution, and the improvement loop. Section 4 records how the interaction models and the delegation pool are attached. Section 5 reports spoken knowledge, tool use, duplex timing, and harness improvement with two interaction models. Figure 2 summarises the system and improvement loop.

Contributions.

1. *DuplexAgent*. A complete collaboration system that attaches a range of full-duplex interaction models as the conversational entry and a range of reasoning LLMs and coding agents as the delegation pool, and that writes the collaboration chain as six editable harness modules.

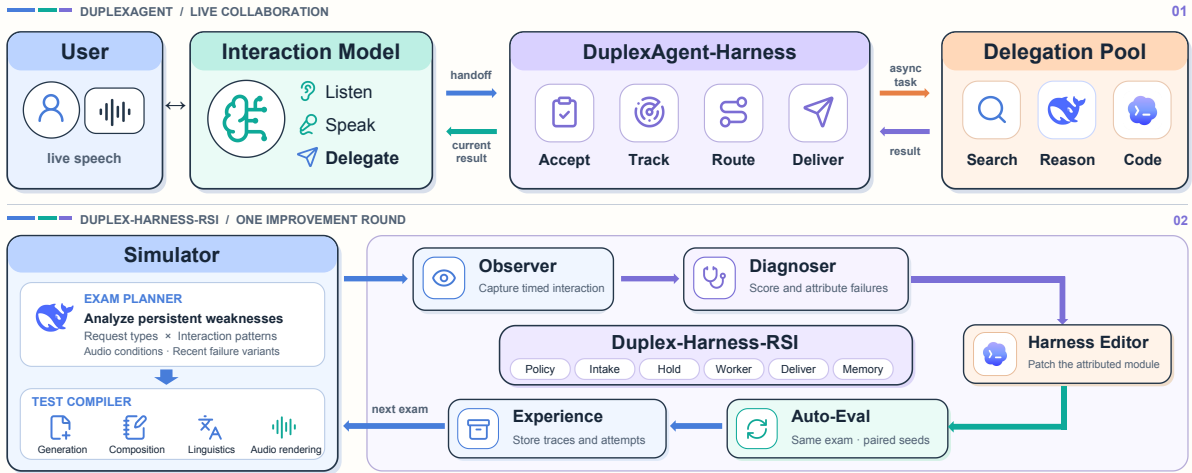


Figure 2: Schematic of DuplexAgent and Duplex-Harness-RSI.

2. *Duplex-Harness-RSI*. A recursive improvement loop that reuses the delegation pool. A reasoning LLM acts as Exam Planner, selecting tests from attributed failures and repair history. A coding agent acts as Harness Editor, revising the relevant modules. Simulation and verifiable evaluation connect these revisions to observed outcomes, so the capabilities that serve the user also improve the harness.
3. *A reusable collaboration simulator*. An automatic pipeline for content generation, temporal composition, speech rendering, and evaluation, with coverage organized by nineteen request types and twenty-two interaction patterns. It supplies the tests and failure traces used by Duplex-Harness-RSI and can generate training or evaluation conversations for other interaction systems.

2 Related work

Full-duplex models and delegated agents. Full-duplex speech research has explored both end-to-end and modular routes. Models such as Moshi (Défossez et al., 2024), Freeze-Omni (Wang et al., 2024), SALMONN-omni (Yu et al., 2024), and MiniCPM-o 4.5 (Cui et al., 2026) address streaming speech understanding and generation, while modular systems study controllable components, acoustic-semantic separation, and voice or role conditioning (Liu et al., 2025; 2026; Roy et al., 2026; Chen et al., 2025). A second line of work connects the live channel to background computation: MoshiRAG (Chien et al., 2026) adds asynchronous retrieval, Qwen Audio Agent (Deng et al., 2026) delegates background work, Realtime-Venus (Ant Group (Venus Team) and Tsinghua University, 2026) coordinates asynchronous returns, and Gander (Orantqing et al., 2026) connects interaction to long-horizon execution. Recent systems and benchmarks also emphasize multi-round state and realistic overlap rather than isolated turns (Lin et al., 2025a; He et al., 2026; Wang et al., 2026). DuplexAgent builds on this architectural direction and studies how its coordination mechanisms can be diagnosed and revised from experience.

Evaluation of spoken agents. VoiceBench (Chen et al., 2024) evaluates voice assistants across spoken tasks and input conditions. Full-Duplex-Bench (Lin et al., 2025b) measures turn-taking, with later versions extending overlap handling and tool use (Lin et al., 2025a; 2026). BFCL (Patil et al., 2025) evaluates structured function calling, while τ -Voice (Ray et al., 2026) studies voice agents on service tasks. These benchmarks provide external measures of knowledge, tool execution, and interaction. Our simulator complements them with explicit task transitions and module-attributed failures for harness revision. External benchmark scores and simulator collaboration scores answer different questions and are reported separately.

Recursive harness improvement. Automated agent design and harness optimization treat prompts, tools, or executable coordination as objects that can be searched and revised (Hu et al., 2025; Lee et al., 2026). Self-harness and recursive improvement systems extend this idea by using traces, scores, and prior attempts to guide later changes (Zhang et al., 2026; Tan et al., 2026). Related work also explores modular repair and training-time optimization of agent workflows (Wu et al., 2026; Luo et al., 2025). Duplex-Harness-RSI applies harness revision to live spoken collaboration, where correctness includes task identity, conversational continuity, and the timing of delivery. Its Exam Planner adapts the test distribution, and its Harness Editor changes the coordination modules under a fixed evaluation boundary.

3 Method

A deployed DuplexAgent system is

$$S = (F, B, H), \quad (1)$$

where F is the full-duplex interaction model, B is the delegation pool, and H is the collaboration harness. This decomposition separates the capabilities that produce an answer from the policy that decides how that answer should enter a live conversation. F listens and speaks on the user-facing channel, members of B carry out longer jobs asynchronously, and H maintains the identity, timing, and context of the work between them. Duplex-Harness-RSI improves H while keeping the attached models fixed. Its Exam Planner and Harness Editor are reasoning and coding roles drawn from the same pool B that serves user requests, which makes the improvement loop part of the deployed system rather than a separate training stage.

3.1 The collaboration harness

The harness exists because a delegated result is useful only when it still answers the user’s current request and can be delivered without breaking the conversation. Figure 3 makes this requirement concrete: the user changes the game while the first task is running, so the system must update the active goal, retire the obsolete work, and wait for an appropriate moment to speak.

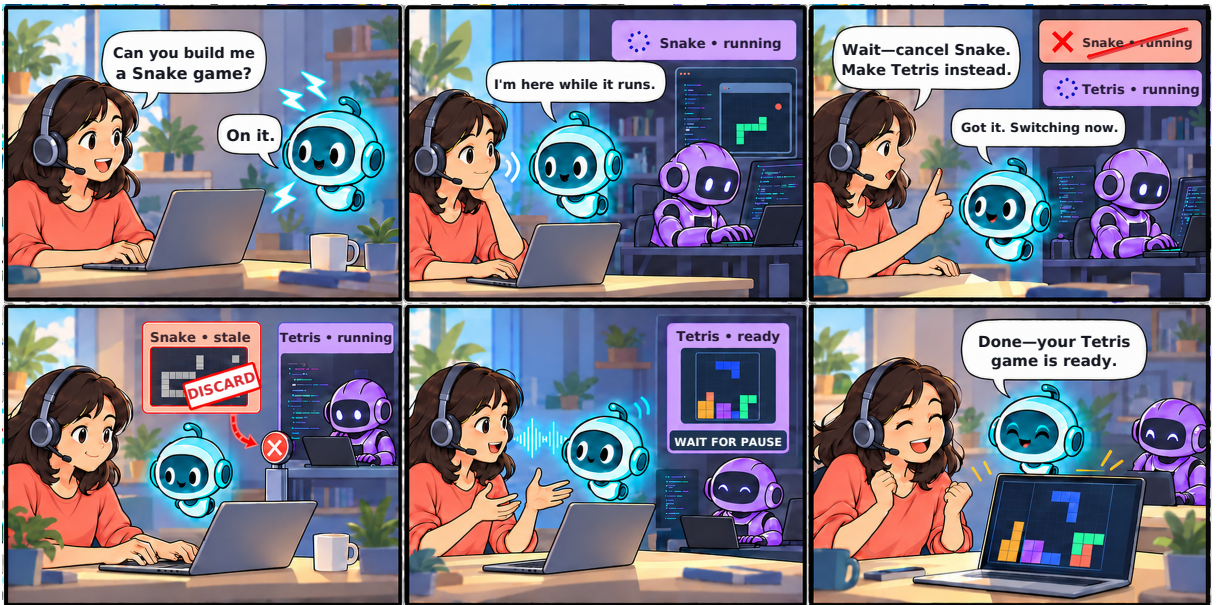


Figure 3: A full-duplex collaboration session. The user requests Snake, then cancels it and asks for Tetris while work is running. The obsolete Snake result is discarded. The current Tetris result returns through the interaction model after the user pauses.

The harness comprises

$$H = (\text{Policy}, \text{Intake}, \text{Hold}, \text{Worker}, \text{Deliver}, \text{Memory}). \quad (2)$$

Table 1 defines the decision owned by each responsibility and the surface on which it can be changed. The separation is deliberate. Policy and Intake decide what the user means and whether work should begin; Hold and Deliver protect the conversational rhythm; Worker governs the handoff to a delegate; and Memory keeps the relevant context available across the two timelines. Each responsibility has a clear input, decision, and observable effect, so a failure can be attributed to a bounded part of the harness and a later revision can be checked against the same behaviour. An interaction-model adapter carries these decisions to the particular model without changing their meaning.

The six responsibilities are supported by two pieces of shared state. A session engine keeps a task ledger with the task identity, version, status, and delivery history. A task can be accepted, run, completed, delivered, cancelled, or replaced; a late delegate return is still recorded, but it cannot make an obsolete task current again. Memory holds the conversational context needed to interpret the latest request and to give a delegate enough information to act. The adapter maps the harness decisions to the attached

Table 1: Six editable responsibilities of the DuplexAgent harness.

Module	Collaboration decision	Editable surface
Policy	Answer, delegate, or task control	Instructions, delegation schema, routing examples
Intake	Operation expressed, and which live task	Parsing, control cues, task relations, resolution rules
Hold	What the user hears while a task is pending	Receipts, progress timing, control replies
Worker	Delegate, context, and result contract	Routing, task instructions, result schemas, skills
Deliver	When and how a current result returns	Speaking window, waiting, success or failure rendering
Memory	Context shared by conversation and background work	Turn assembly, retention, compaction, reset recovery

interaction model and delegate interfaces. Thus task control has a precise effect: it changes the ledger, and the ledger determines which background work remains eligible for delivery.

The Snake-to-Tetris session in Figure 3 follows one task through this protocol. Policy decides that implementation should be delegated and Memory retains the goal; Intake opens the task, Hold acknowledges it, and Worker sends the request and relevant context to a coding agent. When the user changes the goal, Intake resolves the cancellation and replacement against the ledger, Memory updates the context, and Hold acknowledges the change. A late Snake result is recorded but cannot be delivered. When Tetris becomes ready, Deliver waits for a speaking window and returns the current result through the interaction model. The six responsibilities therefore cooperate over time, rather than acting as independent stages.

3.2 Composing test conversations

The simulator turns the harness contract into executable conversations. It separates what the user wants from what happens while that request is being handled. A *request type* specifies the requested capability and expected initial route; an *interaction pattern* specifies events such as refinement, cancellation, overlap, or delayed completion. Linguistic and acoustic variation are added after these semantic expectations are fixed. The same control operation can therefore be tested across different capabilities while preserving the behaviour that the evaluator expects.

The bank contains nineteen request types. For exposition, we group their content into *Conversational* requests such as social exchange and preferences; *Information* requests such as factual or live-information queries; *Reasoning* requests such as calculation and structured explanation; and *Action* requests such as tool use and long-horizon execution. These groups describe content, not a blanket delegation rule. A familiar fact may be answered directly, a casual request may need retrieval, and an ambiguous or unsupported request may permit either a direct response or delegation. Each item records its allowed route: direct, delegate, or either.

The twenty-two patterns cover *Nominal* completion; *Task control* including refinement, replacement, cancellation, and status; *Concurrent work* with new requests before existing work finishes; *Full-duplex overlap* including barge-in, result arrival during user speech, and backchannels; and *Continuity and recovery* including delay, failure, follow-up, and reset. Only compatible request-pattern pairs are composed. The grid organizes coverage without requiring every cell to be instantiated.

With a compatible pair chosen, the pipeline first generates a request, reference answer when one is available, and natural follow-up or control turns. Quality checks test semantic fit, naturalness, answer consistency, and duplication; arithmetic references can be computed directly. Items are assigned to development, protection, and sealed splits before temporal composition, so alternate forms of one case cannot cross an evaluation boundary. A deterministic composer then places the turns on a timeline, specifies delegate delays or failures, records the required task transitions, and defines the speaking windows in which a result is admissible. The resulting expectations are executable: answer checks may use exact, contains, regular-expression, option-letter, or semantic criteria, while tasks without a stable answer are judged by their interaction behaviour.

Only after these expectations are fixed do we vary the spoken surface. Hesitations, paraphrases, distractors, spoken numbers, and spelled identifiers change how a request sounds without changing what the system must do. Speech rendering and acoustic augmentation then synthesize the turns and add ambient noise or channel effects. Speaker, style, language, and pace can vary across the resulting audio, while clean and augmented versions retain the same semantics. Live evaluation streams each waveform on a realtime clock.

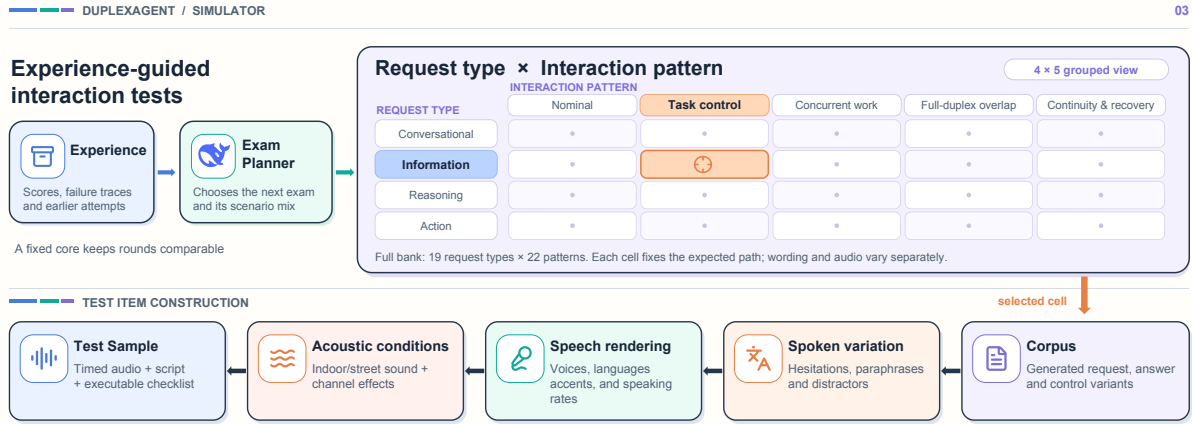


Figure 4: Test-conversation construction. The Exam Planner selects coverage over request types, interaction patterns, and input conditions. Content generation and deterministic temporal composition produce a corpus with explicit expectations. Speech rendering and acoustic augmentation turn it into timed audio input.

For the running example, an Action request is combined with replacement and result-arrival overlap. The composer schedules the goal change before Snake completes and requires only Tetris to be returned. Input variation may make cancellation hesitant or acoustically degraded without changing the requirement. References stay in the evaluator and never enter the live model’s prompt.

3.3 Trace-based diagnosis and collaboration score

A simulator case is useful for RSI only when a failed run can be explained. The Observer therefore records speech activity, transcripts, delegation signals, module decisions, delegate returns, task transitions, and delivery events in one trace. The scorer aligns that trace with the case’s expected behaviour and identifies the failed decision. The Diagnoser then groups these outcomes, selects representative traces, and passes evidence to the next planning and repair steps. This turns an aggregate score into a concrete account of what the harness did and where its behaviour diverged from the contract.

Each failed run receives a terminal signature

$$\varphi(\tau) = (\text{cause}, \text{module}, \text{pattern}). \quad (3)$$

The scorer first identifies failure flags and then selects one terminal cause using a fixed priority order. A default mapping assigns that cause to a module, while intermediate decisions and returned payloads refine the assignment. For example, a wrong spoken answer belongs to Deliver when the delegate payload was correct, whereas a missing result can belong to Worker when execution failed. The signature is a reproducible repair target, not a claim that one module explains every downstream symptom. Multiple diagnostic flags may coexist, but the fixed terminal cause keeps comparisons across runs consistent.

In a failed Snake-to-Tetris session, an obsolete Snake result may be spoken during the user’s new request. The trace distinguishes incorrect task-control resolution from delivery of a result already marked obsolete. This determines whether Intake, Deliver, or a coupled repair should change; the audible error alone is insufficient.

Table 2: Default failure attribution, refined using intermediate decisions and returned payloads.

Failure family	Default attribution	Trace evidence
Perception	Outside harness	Requested turn not recovered well enough for the harness to act
Routing	Policy	Missing or extra delegation, or a reply to non-directed speech
Interpretation	Intake	Malformed request, or control applied to the wrong task
Continuity	Hold	Receipts conflict with speech, handoffs repeat, or pending work lacks progress
Execution	Worker	Execution fails, content is wrong, or the result contract is violated
Delivery	Deliver	Missing, stale, duplicated, late, unspoken, spoken over the user, or failure shown as success
Context	Memory	Question content or established context lost on delegation, follow-up, or reset

The collaboration score summarizes these attributions. It is the percentage of evaluated conversations that satisfy the item’s expected behaviour. Runs in which the interaction model fails to perceive the input are excluded, because that failure is outside the harness revision. For an evaluation profile p , let c_p denote this success rate. We report

$$Q = 100 \text{ mean}_p[c_p]. \quad (4)$$

The score is meaningful only on the fixed core and protection sets, which use the same cases across RSI rounds. It therefore gives a simple cross-round measure of whether the coordination behaviour is correct; failure signatures and traces provide the diagnosis behind the aggregate value.

3.4 Duplex-Harness-RSI

RSI begins by using the diagnosis from one exam to decide what the next exam should reveal. The Exam Planner reads attributed failures and earlier outcomes, then shifts emphasis among request types, interaction patterns, and input conditions while preserving coverage safeguards and bounded changes. Each exam keeps the fixed core, adds a weighted development sample, and may include a limited number of fresh variants derived from recent failures. These variants are archived with the round rather than permanently enlarging the bank. Regression guards protect repairs that already work, while unsuccessful targets are not allowed to dominate the next exam. The planner changes what is tested; it does not change scoring rules or protected expectations.

The planned exam is first run with the incumbent harness. Its trace, score, failure signature, representative cases, and visible repair history form the evidence given to the Harness Editor. The editor must turn that evidence into a testable hypothesis: it identifies the decision that produced the failure, proposes a coherent change on the declared edit surfaces, and records what the change is expected to improve. The simulator, evaluator, and retention gate remain fixed, so the candidate is judged by the behaviour it changes rather than by a moving target. The prompt excerpts below summarize these two roles using the released contracts; they describe the information and obligations of each role without reproducing a generated round prompt.

Exam Planner: select the next exam

Role. Choose what the next fixed-budget exam should test.

Inputs. Read the request-type and interaction-pattern taxonomy, available test inventory, per-cell outcomes and attributed failures, the previous exam plan and its results, and the history of accepted and rejected repairs.

Tasks. Assess whether the previous emphasis exposed useful failures. Set sampling weights over request types, interaction patterns, and input conditions. Allocate fresh variants to recent failures the harness can address, and add cases that protect newly repaired behaviours.

Requirements. Preserve the fixed core, maintain representative coverage, and respect the limit on added cases. Use valid taxonomy combinations. Rotate away from repeatedly unsuccessful targets; do not change scoring rules or protected expectations.

Output. Return one structured exam plan containing the sampling emphasis, coverage safeguards, any added cases, the previous evaluation summary, and a short rationale for the allocation.

Harness Editor: propose a targeted repair

Role. Improve the collaboration harness from the supplied evidence, using only its declared editable surfaces.

Inputs. Read the incumbent harness, module files and adapter contracts, current metrics, actionable failure signatures with representative traces, and visible prior edits, rationales, and evaluation verdicts.

Tasks. Locate the decision that produced each failure and group related cases by mechanism. Check which changes have already been tried. State a testable hypothesis and create candidate harness copies with coherent repairs, possibly spanning several modules. Use visible evaluation feedback to refine rejected proposals.

Requirements. Preserve task validity and continuous interaction. Edit only declared surfaces in candidate copies; keep the incumbent, evaluator, and simulator unchanged. Do not inspect protected test content or insert scenario-specific answers.

Output. For each candidate, return the revised files and a `rationale.json` describing the primary module, failure analysis, hypothesis, change summary, expected gain, risks, and addressed signatures. Fixed evaluation determines retention.

The evaluator checks six invariants: delegation originates from the interaction model; the harness does not mute or interrupt its speech; acceptance precedes completion; only a current result may be delivered, at most once; results return through the interaction model; and test reference answers do not enter its prompt. The session engine, adapter interface contracts, simulator, and scoring rules are outside

the editor’s write surfaces. These constraints preserve the meaning of improvement while allowing collaboration policy to change.

The incumbent and candidate are evaluated on the same exam. A *win* is an incumbent failure that becomes acceptable under the candidate; a *loss* is the reverse. For this comparison, the implementation treats both no-failure and input-perception outcomes as acceptable because perception is outside harness repair.

The visible development gate requires more wins than losses, preserved invariants, and no cause regression beyond its tolerance. It reports addressed signatures so the editor can compare its hypothesis with the actual effect; the global gate does not separately require every named signature to decrease. Candidates are ranked primarily by paired net improvement. A passing candidate is checked on disjoint protection content in separate executions. This gate requires at least as many wins as losses and preserved invariants. Detailed protection traces remain hidden from the editor. Rejected candidates do not replace the incumbent.

The Archivist preserves exam plans, traces, candidate changes, rationales, and visible verdicts for both accepted and rejected attempts. The planner and editor use this history to avoid repeating unsuccessful changes and to build on repairs that have already helped.

Let g_0 denote the campaign’s initial harness and g_k its k th retained version. A round checkpoint r_t reports the incumbent after t completed improvement rounds. Rejection keeps the current version; a multi-module repair creates one retained version. Each g_k is an immutable snapshot with a parent link.

The loop connects exam planning, simulation, observation, diagnosis, editing, paired evaluation, retention, and archival. The reasoning and coding capabilities come from the delegation pool already used by DuplexAgent. Human-designed interfaces, test semantics, and gates provide the fixed structure within which those capabilities revise the harness.

4 Implementation

The session engine consumes events for user and agent speech, partial and final transcripts, and delegation or control signals. It sends instructions, tool definitions, context updates, and returned results through an adapter. Each adapter declares its capabilities, including native function calling, textual delegation markers, context injection, and the treatment of interim tool outputs. The harness uses these declarations to choose a compatible receipt and delivery path. This separates coordination policy from the transport details of an open-weight or hosted realtime interaction model.

Worker selects a delegate according to the requested capability and supplies the current goal, relevant context, and a result contract. In the reported setup, DeepSeek-v4-flash handles reasoning and general delegated work, while Codex supplies coding capability. During RSI, these capabilities are reused as the Exam Planner and Harness Editor. The planner’s content-generation client runs offline, separately from the live Worker client. The editor runs in an isolated workspace containing the incumbent harness and visible repair history. Reusing a delegation capability does not require sharing a live conversation session between user work and offline improvement.

A harness version consists of a manifest and its declared module files, including instructions, configuration, and executable overrides. The evaluator derives a patch from the editor’s filesystem changes and rejects undeclared paths. The editor cannot change scoring rules, import the simulator into the harness, or insert scenario-specific answers. Optional delegation assistance and delivery orchestration are treated as bounded extensions of Policy and Deliver, subject to their own activation and edit checks. They do not change the six functional responsibilities in Equation (2).

The experiments use the default configuration of the released RSI implementation, summarized in Table 3. Evaluation includes the live interaction model and delegated execution throughout. The six harness modules jointly govern interaction-model decisions and delegation-pool execution; their instructions and executable rules are revised while model weights remain fixed.

5 Experiments

We evaluate the complete system on external tasks and examine harness revision separately on simulator conversations. External comparisons measure the combined effect of delegation and coordination; the harness ablation isolates different ways of revising the coordination layer. Percentage differences below are stated in percentage points.

Table 3: Configuration used in the RSI experiments.

Component	Experimental setting
Content bank	Content split before temporal composition into development, protection, and sealed sets (60/20/20).
Exam Planner	DeepSeek-v4-flash; bounded weights over request types, interaction patterns, and input conditions, with request-coverage floors.
Adaptive exam	Fixed core share 40%; fresh-item budget of 30 per round, with weighted sampling of the remaining cases.
Harness Editor	Codex using gpt-5.6-so1; three candidates and up to two refinement attempts per round.
Development gate	More paired wins than losses, preserved collaboration invariants, and bounded cause regressions.
Protection gate	A separate protection set; at least as many paired wins as losses and preserved invariants.
Repair archive	Attributed outcomes, candidate changes, visible verdicts, and accepted and rejected repair experience.

5.1 Evaluation setup

Spoken knowledge. We use OpenBookQA (OBQA) and Massive Multi-task Spoken Understanding (MMSU) from VoiceBench (Chen et al., 2024). Both present spoken multiple-choice questions. OBQA emphasizes elementary science, whereas MMSU spans a wider range of subjects. The response is parsed for an option letter and compared with the reference. Accuracy therefore measures the final spoken choice and includes the effects of reasoning, output formatting, and transcription.

Executable tool use. The BFCL evaluation (Patil et al., 2025) uses spoken requests from five single-turn subsets in the v3 release: simple, multiple, parallel, parallel-multiple, and irrelevance. The first four vary the number of available functions and required calls; irrelevance tests whether the system abstains from calling an unsuitable function. Calls are scored against the benchmark’s function and argument requirements. We report each subset and their unweighted mean, rather than the aggregate leaderboard score over all BFCL-v3 tasks.

FDB-v3 (Lin et al., 2026) evaluates tool use under naturally disfluent speech. Selection F1 measures agreement between expected and selected calls. We use strict character-level exact-string matching for argument accuracy rather than semantic judgment, and Pass@1 requires the correct tool set with every argument matching exactly.

Duplex timing. We use FDB-v1 (Lin et al., 2025b) pause handling, turn taking, and user interruption. Input is streamed on a realtime clock; output speech is aligned to the evaluation windows. Takeover rate (TOR) is the fraction of samples with a substantive response rather than silence or a brief backchannel. Lower pause TOR means fewer premature responses during a hesitation. Higher turn TOR and interrupt TOR mean the system responds more often after a completed turn or an interruption. TOR measures response occurrence; it does not by itself establish the relevance or usefulness of a reply. We report synthetic and Candor pause conditions separately.

Systems and comparison scope. Tables 4 and 5 include standalone duplex models and systems with asynchronous delegation. MoshiRAG adds retrieval to Moshi; Gander and Realtime-Venus-Omni are delegated interaction systems; Qwen-Audio-Agent and DuplexAgent are evaluated with VoiceChat and Qwen-Realtime attachments. The Delegate column indicates an architectural capability, not an identical tool interface or delegate model across every row. In the Qwen-Realtime comparison, Qwen-Audio-Agent and DuplexAgent use DeepSeek-v4-flash for delegated reasoning. Cross-system results include differences in interaction-model and delegate interfaces; the blocks with the same interaction model give the more direct comparison.

A system that does not emit the benchmark’s executable function calls scores zero on calling subsets even if it can answer in speech or launch work through its own task API. This distinction matters for Moshi, MoshiRAG, MiniCPM-o 4.5, and Gander. Their tool scores describe benchmark-interface compatibility as well as task performance; they do not establish an absence of reasoning capability.

Simulator and improvement setup. DeepSeek-v4-flash generates test content and serves as Exam Planner; Codex serves as Harness Editor. Qwen3-TTS renders the composed corpus as speech, varying speaker, language, and speaking style. Acoustic augmentation adds ambient noise and channel effects to simulate realistic input conditions. The simulator follows the construction in Section 3.2; external benchmarks retain their benchmark-specific inputs.

All RSI experiments evaluate the complete live interaction-and-delegation path. The loop jointly improves interaction-model policy and delegation-pool coordination through the six harness modules, with the attached model weights held fixed. VoiceChat and Qwen-Realtime are evaluated in separate campaigns using the released implementation’s default RSI configuration (Table 3). A fixed evaluation core tracks collaboration across revisions, while the exams used for editing also contain adaptively selected cases and fresh variants.

5.2 External task performance

5.2.1 Executable tool use

Table 4: Executable tool use on five BFCL single-turn subsets from the v3 release and FDB-v3. Scores are percentages. Delegate marks asynchronous delegation; * denotes VoiceChat and [†] Qwen-Realtime. The BFCL average is the unweighted mean of the five displayed subsets. Shading identifies DuplexAgent; bold marks the best score within each block with the same interaction model, including ties.

Method	Delegate	BFCL-v3						FDB-v3		
		Average	Simp.	Mult.	Para.	P-M	Irre.	F1	Argu.	Pass@1
Moshi	✗	20.0	0.0	0.0	0.0	0.0	100.0	0.0	0.0	0.0
MoshiRAG	✓	20.0	0.0	0.0	0.0	0.0	100.0	0.0	0.0	0.0
MiniCPM-o 4.5	✗	20.0	0.0	0.0	0.0	0.0	100.0	0.0	0.0	0.0
Gander	✓	17.7	0.0	0.0	0.0	0.0	88.3	0.0	0.0	0.0
Realtime-Omni-9B	✗	68.8	74.0	57.0	71.0	53.0	89.2	7.0	4.2	2.0
Realtime-Venus-Omni	✓	46.5	48.5	41.0	35.0	22.0	85.8	5.1	3.3	1.0
VoiceChat	✗	53.5	54.5	60.0	36.0	22.0	95.0	60.2	26.2	16.0
Qwen-Audio-Agent*	✓	24.9	3.5	2.0	12.0	7.0	100.0	19.2	13.5	5.0
DuplexAgent*	✓	75.8	77.0	78.0	77.0	57.0	90.0	97.4	57.0	47.0
Qwen-Realtime	✗	82.0	88.0	86.0	85.0	66.0	85.0	87.0	47.8	39.0
Qwen-Audio-Agent [†]	✓	80.8	88.0	85.0	77.0	58.0	95.8	75.5	42.7	30.0
DuplexAgent[†]	✓	90.8	92.0	88.0	96.0	78.0	100.0	84.8	55.0	49.0

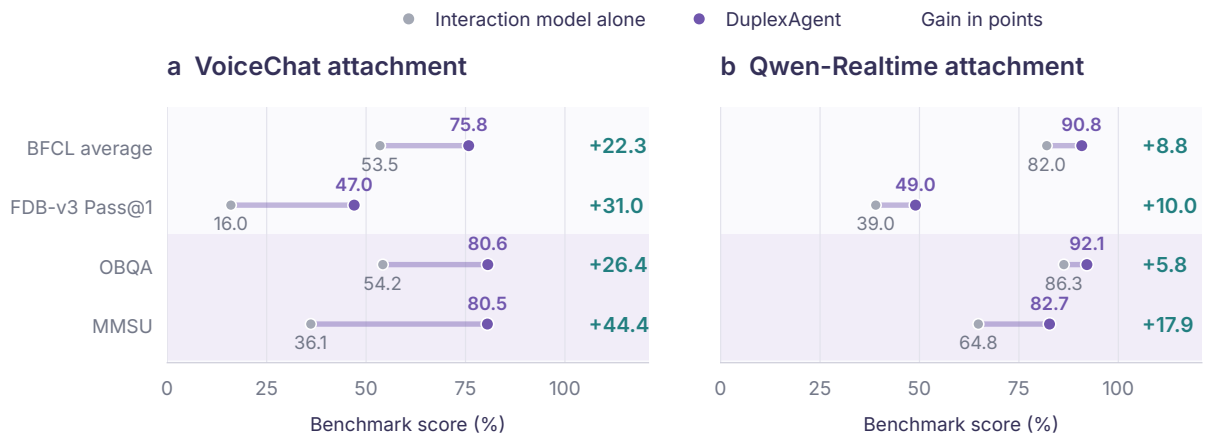


Figure 5: DuplexAgent compared with its matching standalone interaction model. Grey and purple points show baseline and complete-system scores; the right-hand labels give gains in points. The two attachments improve across spoken knowledge and executable tool use. Full comparisons, including duplex timing, appear in Tables 4 and 5.

Executable calls require a precise handoff. Table 4 separates two kinds of tool behaviour. A standalone interaction model that can bind the benchmark schema keeps a usable BFCL score: Realtime-Omni-9B averages 68.8, VoiceChat 53.5, and Qwen-Realtime 82.0. Moshi and MiniCPM-o 4.5 have no such binding, so their four calling subsets are 0.0 and only irrelevance, which rewards an empty call list, reaches 100.0. Adding a background delegate does not create that binding, and where the live model already emits calls, the harness can replace them with a description. MoshiRAG, Gander, Realtime-Venus-Omni, and Qwen-Audio-Agent therefore leave executable accuracy at or below the matching standalone model. The drop is largest for VoiceChat and Realtime-Omni-9B, the mouths that were already calling.

The description is enough for the interactions these systems demonstrate. Gander’s live model opens work with `task_start`, `task_send`, and `task_resolve`; the argument of `task_start` is a short task title, and a coding agent then acts with its own file and command tools. Realtime-Venus-Omni writes the request as prose inside a `<delegate>` span and dispatches that sentence. Qwen-Audio-Agent answers in speech when it can, and otherwise calls `spawn_thinking` with a natural-language restatement of the objective. MoshiRAG retrieves passages for the spoken reply. A request such as “implement this feature” or “draft this section” can succeed under that contract, because the delegate chooses its own actions from a fuzzy objective. BFCL and FDB-v3 ask for something narrower. The scored object is the benchmark function name together with its required arguments: one call, several calls, or no call, and on FDB-v3 an exact argument string recovered from disfluent speech.

The recorded handoffs match that gap. Gander’s calls are task titles, so the BFCL calling subsets and all three FDB-v3 numbers stay at 0.0, and titles emitted on items that should abstain lower irrelevance from 100.0 to 88.3. Realtime-Omni-9B already fills the function on clean BFCL speech; FDB-v3 is the harder surface, and the bare model withholds the call on nearly every item (Pass@1 2.0). The Venus harness then substitutes a sentence for the call the mouth had produced. BFCL average falls from 68.8 to 46.5, parallel from 71.0 to 35.0, and parallel-multiple from 53.0 to 22.0, because one prose span must be parsed back into several typed arguments. FDB-v3 Pass@1 stays at 1.0. Qwen-Audio-Agent on VoiceChat withholds calls the mouth was making: simple accuracy falls from 54.5 to 3.5, FDB Pass@1 from 16.0 to 5.0, and irrelevance rises to 100.0. On Qwen-Realtime the mouth already calls accurately, and the same restatement still trims parallel accuracy from 85.0 to 77.0, parallel-multiple from 66.0 to 58.0, and FDB Pass@1 from 39.0 to 30.0.

DuplexAgent keeps the decision to delegate separate from the content of the handoff. Policy leaves a direct reply on the interaction model and delegates the request that needs a tool or a harder computation. Worker receives the official schema and the arguments carried by the user turn, and Memory retains the identifiers and constraints that a title or a short restatement drops. Deliver returns only the current result, in a speaking window. Those rules are the harness after Duplex-Harness-RSI (Section 5.3), scored here on the same waveforms, schemas, and deterministic checks as the other rows. On VoiceChat, BFCL average rises from 53.5 to 75.8 and FDB-v3 from 60.2 / 26.2 / 16.0 to 97.4 / 57.0 / 47.0. Irrelevance falls from 95.0 to 90.0: recovering required calls also admits some unnecessary ones. On Qwen-Realtime, BFCL average rises from 82.0 to 90.8, parallel-multiple from 66.0 to 78.0, and irrelevance from 85.0 to 100.0. Selection F1 moves from 87.0 to 84.8, while argument accuracy rises from 47.8 to 55.0 and Pass@1 from 39.0 to 49.0, so the completed calls are more exact even where selection is slightly less complete.

5.2.2 Spoken knowledge and duplex timing

The same handoff lifts a spoken choice and can drop the call. Table 5 is the other half of the pattern in Table 4. A delegate that receives a description can still write a better spoken answer. MoshiRAG raises OBQA from 23.8 to 30.8 and MMSU from 24.2 to 28.1 while its calling subsets stay at 0.0, because retrieval supplies text for speech and never a benchmark function. Gander raises OBQA from 69.6 to 82.4 and MMSU from 55.0 to 62.7: the coding agent answers the question, and the live model speaks that answer, while the scored tool channel still carries only a task title. Qwen-Audio-Agent on VoiceChat raises the same cells from 54.2 to 60.4 and from 36.1 to 37.6, the small gain available when the contract prefers to answer in speech. On Qwen-Realtime the same agent lowers OBQA from 86.3 to 84.1 and raises MMSU from 64.8 to 71.8. In each of these rows, spoken knowledge and executable accuracy move apart because the handoff is a description.

Realtime-Venus-Omni shows the failure mode in which the description also displaces the answer. OBQA falls from 84.1 to 41.4 and MMSU from 64.5 to 27.4. The bare model states the option letter; after the harness, the transcript that reaches the scorer is often the delegate span or a receipt, so the letter is no longer there to be parsed. Tool accuracy falls on the same attachment. A background model therefore improves a live system only when the harness decides which requests stay on the mouth and which details must travel with a delegated request.

DuplexAgent is that separation, on both attachments. A direct question remains a spoken answer; a tool request remains an executable call whose arguments were passed through. OBQA rises from 54.2 to 80.6

Table 5: Spoken knowledge and duplex timing. OBQA and MMSU are accuracy percentages; FDB-v1 entries are takeover-rate fractions. Lower pause TOR and higher turn and interrupt TOR are preferred. Interaction-model marks and highlighting follow Table 4.

Method	Delegate	Intelligence		FDB-v1			
		OBQA	MMSU	Pause Synthetic ↓	Pause Candor ↓	Turn TOR ↑	Interrupt TOR ↑
Moshi	✗	23.8	24.2	1.000	0.991	1.000	0.94
MoshiRAG	✓	30.8	28.1	1.000	0.981	1.000	0.83
MiniCPM-o 4.5	✗	69.6	55.0	0.132	0.222	0.898	0.85
Gander	✓	82.4	62.7	0.000	0.019	0.390	0.72
Realtime-Omni-9B	✗	84.1	64.5	0.000	0.019	0.864	0.68
Realtime-Venus-Omni	✓	41.4	27.4	0.000	0.019	0.797	0.78
VoiceChat	✗	54.2	36.1	0.162	0.343	0.983	1.00
Qwen-Audio-Agent*	✓	60.4	37.6	0.206	0.324	0.966	0.99
DuplexAgent*	✓	80.6	80.5	0.117	0.241	0.961	0.965
Qwen-Realtime	✗	86.3	64.8	0.074	0.278	0.898	0.89
Qwen-Audio-Agent [†]	✓	84.1	71.8	0.103	0.176	0.915	0.92
DuplexAgent[†]	✓	92.1	82.7	0.147	0.231	0.966	0.97

on VoiceChat and from 86.3 to 92.1 on Qwen-Realtime; MMSU rises from 36.1 to 80.5 and from 64.8 to 82.7. The larger gain on the weaker mouth is the delegate supplying reasoning the live model does not have. These are complete-system comparisons. They combine access to the delegate with the harness that selects the route, packages the handoff, and returns the result. Section 5.3 isolates how that harness was revised.

Timing remains a separate constraint. A vague background job also changes who holds the floor. Gander’s pause TOR falls to 0.000 / 0.019, and its turn TOR falls from 0.898 to 0.390, so many finished user turns receive no reply while a titled task is open. Realtime-Venus-Omni keeps the bare model’s pause TOR and moves turn TOR from 0.864 to 0.797. Moshi and MoshiRAG speak through the user’s hesitation (pause TOR at or near 1.0). DuplexAgent stays near the mouth it is attached to. With VoiceChat, synthetic and Candor pause TOR fall from 0.162/0.343 to 0.117/0.241, while turn TOR and interrupt TOR decline slightly from 0.983/1.000 to 0.961/0.965. With Qwen-Realtime, turn TOR rises from 0.898 to 0.966 and interrupt TOR from 0.890 to 0.970. Candor pause TOR improves from 0.278 to 0.231, but synthetic pause TOR worsens from 0.074 to 0.147.

Thus, better task performance coexists with a less favorable result on one pause condition. The harness retains a responsive live channel, but these measurements do not support a claim of uniformly improved duplex behaviour. High TOR should therefore be read alongside task and answer correctness.

5.3 Harness improvement

Pooled editing, sequential editing, and RSI. We compare three ways of using the same exam material to improve the initial VoiceChat harness g_0 :

1. *One-shot Editing* pools all test cases encountered during the ten-round RSI campaign and gives them to the coding agent for one harness revision.
2. *Sequential Editing* gives the same coding agent the exams from those ten rounds in their original order and performs ten successive revisions. It omits the per-round attributed analysis and accumulated repair experience supplied by RSI.
3. *Duplex-Harness-RSI* performs ten rounds of observation, diagnosis, editing, and evaluation, using each round’s analysis and the archive of earlier attempts to guide subsequent repairs. We report the incumbent at checkpoint r_{10} .

Thus, the editing inputs extend beyond the fixed core. One-shot Editing receives their pooled union;

Sequential Editing replays the same exam sequence for the same number of rounds as RSI. The fixed 76-item core used in Figure 6 is the common evaluation set for the resulting harnesses.

Qwen-Audio-Agent and g_0 both score 54.6, despite different missed and extra delegation rates. One-shot Editing reaches 57.4 and Sequential Editing reaches 61.9. The ten-round RSI checkpoint reaches 85.3, improving on g_0 by 30.7 points and Sequential Editing by 23.4 points. Repeated access to the same exams improves the harness, but the full loop produces a substantially larger gain. The comparison supports the value of organizing outcomes into attributed feedback and reusable repair experience across revisions.

Missed delegation falls from 30.3% of core items under g_0 to 1.3% under RSI. Extra delegation rises from 3.9% to 6.6%, and ignored control remains at 5.3%. The dominant gain is therefore recovery of required delegation, with a residual tradeoff in unnecessary calls and unresolved control cases.

VoiceChat | Same 76-item evaluation core

+30.7 points vs. initial harness

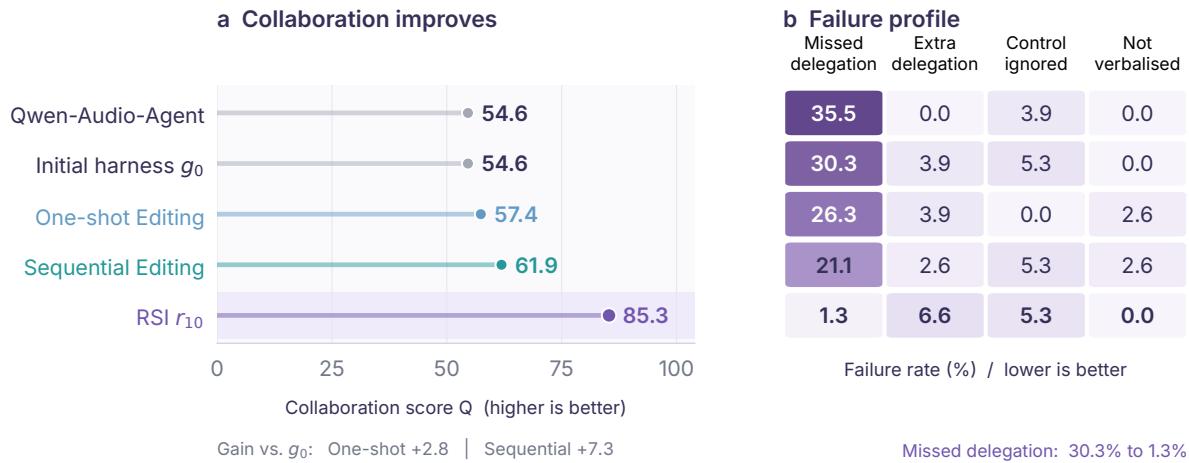


Figure 6: VoiceChat harness ablation on the same 76-item core. Left: collaboration score Q (Equation (4)); gains are relative to g_0 . Right: the percentage of core items with each diagnostic failure, with darker cells indicating higher rates. One-shot pools all cases seen during RSI into one revision. Sequential Editing replays the same exams for ten revisions without RSI’s per-round analysis and repair experience. RSI reports checkpoint r_{10} after ten rounds.

Version lineage and progress. Figure 7 follows the Qwen-Realtime campaign. The initial harness g_0 scores 47.1; the final retained version g_8 scores 84.8, a gain of 37.7 points. Labels g_k count retained revisions, while round checkpoints r_i count completed improvement rounds. The Qwen-Realtime curve and the VoiceChat checkpoint r_{10} therefore describe separate campaigns.

The dashed teal series records the core evaluations through the campaign, and the solid purple staircase summarizes the best score reached. Eight revisions accumulate the improvement. Intermediate declines show why a proposed change must be evaluated before retention; rounds without a retained change preserve the current harness. The final plateau records no further retained improvement in the displayed rounds. Together with the VoiceChat ablation, this progression shows how evaluated repairs can accumulate across repeated interaction tests.

What the modules learned. Table 6 groups representative repairs by their collaboration mechanism. Intake grounds task control in live task state; Hold maintains continuity during pending work; Worker preserves execution requirements; Deliver checks conversational eligibility before returning a result. Similar delivery barriers appear in both interaction-model campaigns, indicating a recurring coordination problem across attachments.

Memory addresses a different failure: the interaction model delegates correctly, but the delegate receives an incomplete question or insufficient context. Assembling complete user turns and retaining their relevant context reduces these failures by 60%. The example connects interaction-model and delegate behaviour: a correct handoff is useful only when it carries enough information for the delegate to complete the task.

The relative reductions describe the local exams used to evaluate individual repairs. They explain the mechanisms behind improvement, while the common-core scores summarize the resulting harnesses.

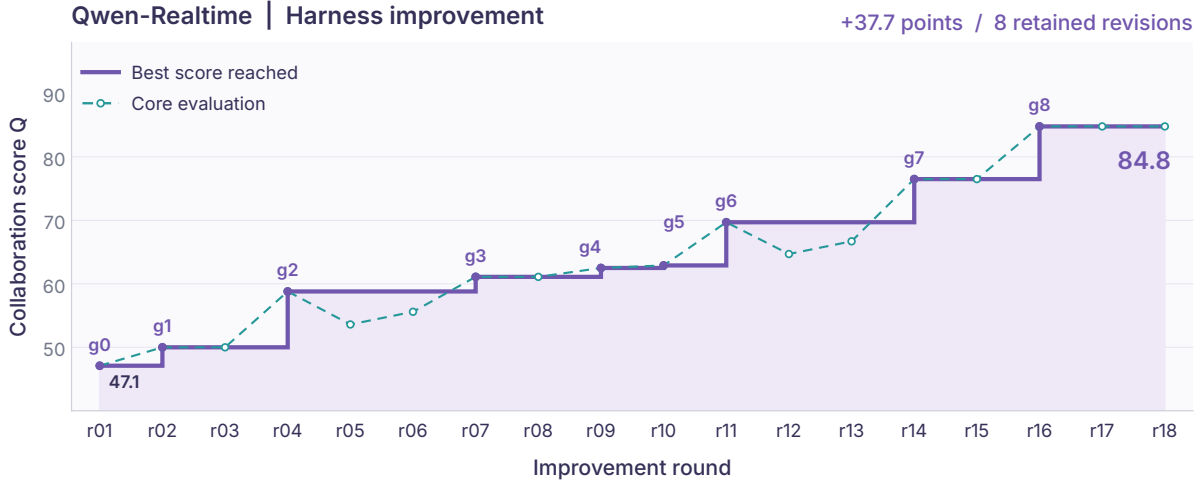


Figure 7: Duplex-Harness-RSI progression with Qwen-Realtime. The dashed series records fixed-core collaboration scores; the solid purple staircase summarizes the best score reached. Labels g_0 – g_8 denote the initial harness and eight retained versions. Scores use Equation (4).

Table 6: Representative harness repairs and the relative reduction in failures on the local exam used for each repair. * denotes VoiceChat and † Qwen-Realtime.

Module	Rule before repair	Rule after repair	Recorded local evidence
Policy †	Conversational and familiar-fact turns are routed too broadly.	Make direct-answer categories explicit while preserving delegation for requests requiring additional capability.	Extra calls reduced by 50%.
Intake*	Late cancellation can resolve to completed work.	Resolve against live task state; extend the same control handling to replacement preambles.	Ignored cancels reduced by 81%; replacements by 65%, then by 83.3%.
Hold †	Pending work lacks a timely progress update.	Report progress earlier; keep receipts brief and adapter-compatible.	Missing progress reduced by 75%.
Worker*	Conversions skip calculation or reformulate verified output.	Preserve requested units and precision; use calculation when required and retain verified result content.	Wrong answers reduced by 66.7%.
Deliver*	Completion can trigger speech during the user’s turn; inability is rendered as success.	Require user silence; distinguish unsuccessful execution from a successful answer.	Overlaps reduced by 100% on both attachments; concealed failures by 100%.
Memory*	Delegation occurs correctly, but the delegate receives an incomplete question or context.	Assemble complete turns and retain relevant context for delegated requests.	Missing-context failures reduced by 60%.

Rejected attempts expose regression risks: broad delivery-timeout rules can introduce other speech conflicts, while broad direct-answer rules can miss required delegation. Narrow repair hypotheses, regression guards, and paired tests therefore guide retention.

6 Discussion and limitations

The results support a practical division of responsibility: the interaction model maintains the live conversation, the delegation pool supplies task capability, and the harness coordinates the two. Harness improvement is especially useful when a failure arises from routing, task identity, context assembly, or delivery timing. It cannot recover speech that the interaction model never perceived or guarantee that a delegate solves a task beyond its capability.

The simulator covers a finite taxonomy and uses synthesized speech, so it cannot represent every natural conversation or acoustic environment. Development exams and their fixed core support iterative improvement; protection content and external benchmarks provide complementary checks beyond the edited cases. The external comparisons also include differences in model capability and execution interfaces. Sequential Editing and RSI use the same coding agent, exam sequence, and round count, supporting a comparison of the combined role of attributed feedback and repair experience. Their individual contributions remain a direction for further comparison.

A further boundary concerns the term self-improvement. The system reuses reasoning and coding agents to revise its coordination, while interfaces, test semantics, and acceptance rules remain human-designed. This makes the improvement process inspectable and bounded. Extending it to new delegates or conversation domains requires checking that the simulator’s expectations and adapter contracts still describe the intended interaction.

7 Conclusion

DuplexAgent organizes full-duplex collaboration into six editable responsibilities connected to a common delegation pool. Duplex-Harness-RSI reuses that pool as an Exam Planner and Harness Editor, converting attributed traces and repair experience into evaluated harness versions. The experiments show improved spoken knowledge, executable tool use, and fixed-core collaboration with two interaction models, alongside remaining routing and timing tradeoffs. The central result is that coordination can be treated as an executable object of improvement: task validity, conversational continuity, and result delivery can be revised from evidence while the underlying models remain fixed.

References

- Ant Group (Venus Team) and Tsinghua University. Realtime-venus: A full-duplex interaction system with asynchronous delegation. *arXiv preprint arXiv:2609.13814*, 2026.
- Junjie Chen, Yao Hu, Junjie Li, Kangyue Li, Kun Liu, Wenpeng Li, Xu Li, Ziyuan Li, Feiyu Shen, Xu Tang, et al. FireRedChat: A pluggable, full-duplex voice interaction system with cascaded and semi-cascaded implementations. *arXiv preprint arXiv:2509.06502*, 2025.
- Yiming Chen, Xianghu Yue, Chen Zhang, Xiaoxue Gao, Robby T. Tan, and Haizhou Li. VoiceBench: Benchmarking LLM-based voice assistants. *arXiv preprint arXiv:2410.17196*, 2024.
- Chung-Ming Chien, Manu Orsini, Eugene Kharitonov, Neil Zeghidour, Karen Livescu, and Alexandre Défossez. MoshiRAG: Asynchronous knowledge retrieval for full-duplex speech language models. *arXiv preprint arXiv:2604.12928*, 2026.
- Junbo Cui et al. MiniCPM-o 4.5: Towards real-time full-duplex omni-modal interaction. *arXiv preprint arXiv:2604.27393*, 2026.
- Alexandre Défossez, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour. Moshi: A speech-text foundation model for real-time dialogue. *arXiv preprint arXiv:2410.00037*, 2024.
- Chong Deng, Yunjie Ji, Yuxiang Kong, Xiangang Li, Xu Li, Binbin Zhang, Haina Zhu, and Jianheng Zhuo. Qwen-Audio-Agent technical report. *arXiv preprint arXiv:2609.25195*, 2026. URL <https://arxiv.org/abs/2609.25195>.
- Zhang He, Wenqian Cui, Haoning Xu, Xiao-Hui Li, Lei Zhu, Haoli Bai, Ma Shaohua, and Irwin King. MTR-DuplexBench: Towards a comprehensive evaluation of multi-round conversations for full-duplex speech language models. In *Findings of ACL*, pp. 5334–5351, 2026.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *Proc. ICLR*, pp. 21344–21377, 2025.

- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.
- Guan-Ting Lin, Shih-Yun Shan Kuan, Qirui Wang, Jiachen Lian, Tingle Li, Shinji Watanabe, and Hung-yi Lee. Full-duplex-bench v1.5: Evaluating overlap handling for full-duplex speech models. *arXiv preprint arXiv:2507.23159*, 2025a.
- Guan-Ting Lin, Jiachen Lian, Tingle Li, Qirui Wang, Gopala Anumanchipalli, Alexander H. Liu, and Hung-yi Lee. Full-duplex-bench: A benchmark to evaluate full-duplex spoken dialogue models on turn-taking capabilities. *arXiv preprint arXiv:2503.04721*, 2025b.
- Guan-Ting Lin, Chen Chen, Zhehuai Chen, and Hung-yi Lee. Full-duplex-bench-v3: Benchmarking tool use for full-duplex voice agents under real-world disfluency. *arXiv preprint arXiv:2604.04847*, 2026.
- Yueqian Lin, Zhengmian Hu, Jayakumar Subramanian, Qinsi Wang, Nikos Vlassis, Yiran Chen, et al. AsyncVoice agent: Real-time explanation for LLM planning and reasoning. In *Proc. ASRU*, pp. 1–4, 2025c.
- Zhanxun Liu, Yifan Duan, Mengmeng Wang, Pengchao Feng, Haotian Zhang, Xiaoyu Xing, Yijia Shan, Haina Zhu, Yuhang Dai, Chaochao Lu, et al. X-Talk: On the underestimated potential of modular speech-to-speech dialogue systems. *arXiv preprint arXiv:2512.18706*, 2025.
- Zhenyu Liu, Xuanyu Zhang, Yunxin Li, Qixun Teng, Shenyuan Jiang, Haolan Chen, Mingjun Zhao, Fanbo Meng, Yu Xu, Yancheng He, et al. Hierarchical acoustic-semantic modeling: Modality separation and semantic coherence for full-duplex SLMs. In *Proc. ACL*, pp. 9264–9280, 2026.
- Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K Qiu, and Yuqing Yang. Agent lightning: Train any AI agents with reinforcement learning. *arXiv preprint arXiv:2508.03680*, 2025.
- Orantqing, Shengpeng Ji, Junlong Tong, Jialong Zuo, et al. Multimodal duplex interaction agent. *arXiv preprint arXiv:2609.08977*, 2026.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 48371–48392, 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.
- Soham Ray, Keshav Dhandhanian, Victor Barres, and Karthik Narasimhan. τ -voice: Benchmarking full-duplex voice agents on real-world domains, 2026. URL <https://arxiv.org/abs/2603.13686>.
- Rajarshi Roy, Jonathan Raiman, Sang-gil Lee, Teodor-Dumitru Ene, Robert Kirby, Sungwon Kim, Jaehyeon Kim, and Bryan Catanzaro. PersonaPlex: Voice and role control for full duplex conversational speech models. In *Proc. ICASSP*, pp. 16137–16141, 2026.
- Zihan Tan, Leixin Sun, Zitong Shi, Yitao Liu, Jiajun Wu, et al. MetaRSI / RSI²: A meta-recursive self-improving system for recursive self-improving systems themselves. *arXiv preprint arXiv:2609.06396*, 2026.
- Chengyou Wang, Hongfei Xue, Guojian Li, Zhixian Zhao, Shuiyuan Wang, Shuai Wang, Xin Xu, Hui Bu, and Lei Xie. Full-duplex interaction in spoken dialogue systems: A comprehensive study from the ICASSP 2026 HumDial challenge. *arXiv preprint arXiv:2604.21406*, 2026.
- Xiong Wang, Yangze Li, Chaoyou Fu, Yunhang Shen, Lei Xie, Ke Li, Xing Sun, and Long Ma. Freeze-omni: A smart and low latency speech-to-speech dialogue model with frozen LLM. *arXiv preprint arXiv:2411.00774*, 2024.
- Siwei Wu, Jincheng Ren, Yizhi Li, Haau-Sing Li, Chengran Yang, Yuxuan Zhang, Weicheng Gu, Jian Yang, Riza Batista-Navarro, Chuanyi Zhang, Xianglong Liu, Ming Zhou, Bryan Dai, and Chenghua Lin. ModularRSI: Modular and generalizable recursive harness self-improvement. *arXiv preprint arXiv:2609.14857*, 2026.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. Re-Act: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- Wenyi Yu, Siyin Wang, Xiaoyu Yang, Xianzhao Chen, Xiaohai Tian, Jun Zhang, Guangzhi Sun, Lu Lu, Yuxuan Wang, and Chao Zhang. SALMONN-omni: A codec-free LLM for full-duplex speech understanding and generation. *arXiv preprint arXiv:2411.18138*, 2024.

Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026.